AI Data Analytics SaaS

Complete Technical Build Plan & Architecture Guide

A production-ready AI-powered data analytics platform with marketing site, authentication, billing, admin panel, and full analytics stack.

~85	9	12+
Files	Phases	Integrations

Project Directory: /Users/mrinal/Documents/ai-analytics
Date: February 18, 2026
Framework: Next.js 15 · TypeScript · Tailwind CSS 4

Table of Contents

1 Executive Summary & Product Overview

2 Technology Stack & Rationale

3 Project Structure & Architecture

4 Database Schema Design

5 Phase 1: Project Foundation

6 Phase 2: Authentication System

7 Phase 3: Marketing Site

8 Phase 4: Application Dashboard

9 Phase 5: API Routes

10 Phase 6: AI Engine

11 Phase 7: Analytics & Marketing Pixels

12 Phase 8: Admin Panel

13 Phase 9: Polish & Deployment

14 Pricing & Business Model

15 Data Flow Architecture

16 Security Considerations

17 Verification & Testing Checklist

1. Executive Summary & Product Overview

This document outlines the complete technical plan for building an AI-powered Data Analytics SaaS platform. The product enables users to upload datasets in CSV, Excel, and JSON formats, analyze them using AI (powered by Anthropic's Claude API), generate interactive visualizations, and create comprehensive reports through natural language queries.

What the Product Does

Dataset Management

Upload CSV, Excel, and JSON files. The system auto-detects schemas, data types, and generates previews. Datasets are stored securely and linked to user accounts.

AI-Powered Analysis

Chat with your data using natural language. Claude AI analyzes datasets, identifies patterns, suggests insights, and generates statistical summaries automatically.

Smart Visualizations

AI recommends the best chart types for your data. Interactive Recharts-powered visualizations with export capabilities for reports and presentations.

Report Generation

Compile multiple analyses into structured reports. Export as PDF or share via link. Reports include charts, insights, and data summaries.

Target Users

- **Business Analysts** — who need quick insights without writing SQL or Python
- **Product Managers** — who want to explore metrics and generate reports for stakeholders
- **Small Business Owners** — who have spreadsheet data but lack analytical expertise
- **Data Teams** — who want AI-assisted exploration as a complement to their existing tools

Platform Architecture Overview

The platform is divided into four main sections, each with its own layout:

- **Marketing Site** (marketing) — Public-facing landing page, pricing, and SEO-optimized content
- **Auth Pages** (auth) — Login, signup, password reset with centered minimal layout

- **App Dashboard** (app) — Protected workspace with sidebar navigation for authenticated users
- **Admin Panel** (admin) — Administrative controls for user management, revenue tracking, and platform health

2. Technology Stack & Rationale

Every technology choice is deliberate, optimized for a serverless-first SaaS that can scale from zero to production with minimal operational overhead.

Layer	Technology	Rationale
Framework	Next.js 15 (App Router)	Server-side rendering for marketing SEO, API routes for backend, React Server Components for performance, middleware for auth guards. App Router's route groups enable separate layouts per section.
Language	TypeScript	Type safety across the full stack — Prisma generates types from DB schema, API routes have typed request/response, components have typed props. Catches bugs at compile time rather than production.
Styling	Tailwind CSS 4 + shadcn/ui	Tailwind 4 uses CSS-first config (no tailwind.config.ts). shadcn/ui provides accessible, customizable components that are copied into the project (not a dependency), giving full control. Dark mode via CSS variables.
Auth	Auth.js v5 (NextAuth)	JWT sessions for serverless (no session DB lookups), supports Google OAuth + email/password + magic links. Prisma adapter auto-manages Account/Session tables. Built-in CSRF protection.
Database	PostgreSQL (Neon) + Prisma	Neon provides serverless Postgres with auto-scaling and a generous free tier. Prisma ORM gives type-safe queries, auto-generated migrations, and a visual studio for DB exploration.
Payments	Stripe	Industry standard for SaaS billing. Handles subscription lifecycle, payment collection, invoicing, tax calculation, and customer portal. Webhook-driven architecture ensures data consistency.
AI	Anthropic Claude API	200K context window handles large datasets. Structured output for chart configs. Superior instruction following for data analysis prompts. Cost-effective for analytical workloads.
Analytics	GTM + GA4 + PostHog + Pixels	GTM as universal container — add/remove ad pixels without code changes. PostHog for product analytics (session replay, funnels, feature flags). Dual tracking ensures both marketing and product teams have the data they need.

Layer	Technology	Rationale
File Storage	Vercel Blob	Serverless file storage with CDN distribution. No S3 configuration needed. Integrates natively with Next.js API routes.
Charts	Recharts	React-native charting library. Declarative API makes it easy for AI to generate chart configurations. Supports all common chart types (bar, line, area, pie, scatter).
Deployment	Vercel	Zero-config deployment from Git. Edge middleware for auth, built-in CDN, automatic HTTPS, preview deployments for every PR.

Why Serverless-First?

Every component (Neon DB, Vercel Functions, Vercel Blob, Stripe) scales to zero when idle and auto-scales under load. This means **$0/month cost at zero users** and linear cost scaling as the user base grows. No servers to manage, no containers to orchestrate — ideal for a launch-stage SaaS.

3. Project Structure & Architecture

Directory Layout

The project follows Next.js 15 App Router conventions with route groups for layout isolation:

```
ai-analytics/
├── prisma/
│   └── schema.prisma              # Database schema (7 models)
├── public/
│   └── images/                    # Static assets, logos, OG images
├── src/
│   ├── app/
│   │   ├── (marketing)/           # Route group: public marketing pages
│   │   │   ├── page.tsx           # Landing page (/)
│   │   │   ├── pricing/page.tsx   # Pricing page (/pricing)
│   │   │   └── layout.tsx         # Marketing Navbar + Footer
│   │   ├── (auth)/                # Route group: auth pages
│   │   │   ├── login/page.tsx     # Login (/login)
│   │   │   ├── signup/page.tsx    # Signup (/signup)
│   │   │   ├── forgot-password/page.tsx
│   │   │   └── layout.tsx         # Centered, minimal layout
│   │   ├── (app)/                 # Route group: protected app
│   │   │   ├── dashboard/page.tsx # Overview (/dashboard)
│   │   │   ├── datasets/page.tsx  # Dataset management (/datasets)
│   │   │   ├── analyze/page.tsx   # AI workspace (/analyze)
│   │   │   ├── reports/page.tsx   # Report gallery (/reports)
│   │   │   ├── settings/page.tsx  # User settings (/settings)
│   │   │   ├── billing/page.tsx   # Billing management (/billing)
│   │   │   └── layout.tsx         # Sidebar + Topbar shell
│   │   ├── (admin)/               # Route group: admin panel
│   │   │   ├── admin/page.tsx     # Admin dashboard (/admin)
│   │   │   ├── admin/users/page.tsx  # User management (/admin/users)
│   │   │   └── layout.tsx         # Admin sidebar
│   │   ├── api/                   # API routes (serverless functions)
│   │   │   ├── auth/[...nextauth]/route.ts
│   │   │   ├── stripe/            # Checkout, portal, webhook
│   │   │   ├── ai/                # Analyze, query
│   │   │   ├── datasets/          # CRUD + upload
│   │   │   └── admin/             # Users, stats
│   │   ├── layout.tsx             # Root layout (providers, GTM, metadata)
│   │   └── globals.css            # Tailwind 4 base styles
│   ├── components/
│   │   ├── ui/                    # shadcn/ui primitives
│   │   ├── marketing/             # Hero, Features, Testimonials, Footer
│   │   ├── dashboard/             # Stats, charts, activity feed
│   │   ├── datasets/              # Dropzone, table, preview
│   │   ├── analyze/               # Chat interface, chart renderer
│   │   ├── auth/                  # Login/signup forms
│   │   └── shared/                # Navbar, Sidebar, UserMenu, ThemeToggle
│   ├── lib/
```

```
│   │   ├── auth.ts                # Auth.js v5 configuration
│   │   ├── prisma.ts              # Prisma client singleton
│   │   ├── stripe.ts              # Stripe helpers
│   │   ├── ai.ts                  # Claude API service
│   │   ├── upload.ts              # File parsing (CSV/Excel/JSON)
│   │   └── analytics/             # GTM, PostHog, unified events
│   ├── hooks/                     # React hooks
│   ├── types/                     # TypeScript type definitions
│   └── middleware.ts              # Route protection middleware
├── .env.example
├── next.config.ts
├── tailwind.config.ts
├── package.json
└── tsconfig.json
```

Route Groups Explained

Parenthesized folders like `(marketing)` create **route groups** in Next.js App Router. They organize routes with shared layouts without affecting the URL path. So `(marketing)/pricing/page.tsx` maps to `/pricing`, not `/marketing/pricing`. This lets each section have its own `layout.tsx` — marketing gets a navbar+footer, the app gets a sidebar, auth gets a centered card.

4. Database Schema Design

The database uses 7 core models managed by Prisma ORM. The schema is designed for a multi-tenant SaaS with subscription-based access control.

Entity Relationship Overview

```
User ──┬── Subscription (1:1) ─── Stripe Customer/Subscription
       ├── Dataset[] (1:many) ─── Report[] (1:many per dataset)
       ├── Account[] (1:many) ─── OAuth providers (Google, etc.)
       ├── Session[] (1:many) ─── Active sessions
       └── AuditLog[] (1:many) ─── Action history
```

Model Details

User Model

Central entity. Stores identity, role, and links to all user-owned resources.

Field	Type	Description
id	String (CUID)	Primary key, collision-resistant unique ID
name	String?	Display name (optional for OAuth users)
email	String (unique)	Primary identifier, used for login
password	String?	bcrypt hash, null for OAuth-only users
role	Enum (USER/ADMIN)	Access control — guards admin panel routes
emailVerified	DateTime?	Set when email is confirmed
image	String?	Avatar URL from OAuth provider

Subscription Model

1:1 with User. Tracks Stripe subscription state and usage limits per billing period.

Field	Type	Description
stripeCustomerId	String (unique)	Links to Stripe customer object
stripeSubscriptionId	String? (unique)	Active subscription ID (null = free tier)

Field	Type	Description
plan	Enum (FREE/PRO/ENTERPRISE)	Current subscription tier
status	String	active, past_due, canceled, trialing
currentPeriodEnd	DateTime?	When current billing period expires
queriesUsed	Int	AI queries consumed this period
queriesLimit	Int	Max queries allowed (10/500/unlimited by tier)

Dataset Model

Stores metadata about uploaded files. Actual file content is stored in Vercel Blob.

Field	Type	Description
name	String	User-facing name (editable)
fileName	String	Original file name on upload
fileUrl	String	Vercel Blob URL for the stored file
format	String	csv, xlsx, or json
rowCount	Int	Number of data rows (for display + limits)
columns	Json	Schema: column names, inferred types, sample values

Report Model

Stores AI-generated analysis results including chart configurations and insights.

Field	Type	Description
title	String	Auto-generated or user-set report title
content	Json	Structured analysis: insights[], charts[], summary
datasetId	String (FK)	The dataset this report was generated from
userId	String (FK)	Owner of the report

AuditLog Model

Immutable log of significant user and system actions for compliance and debugging.

Field	Type	Description
action	String	e.g., "dataset.upload", "ai.query", "subscription.upgrade"
details	Json?	Action-specific metadata (file size, query text, etc.)
userId	String? (FK)	Who performed the action (null for system events)
ipAddress	String?	Request origin IP

Phase 1: Project Foundation

~15 files · Project setup, dependencies, database, middleware

1.1 Project Initialization

Create the Next.js 15 project with TypeScript, Tailwind CSS, and App Router enabled:

```
npx create-next-app@latest ai-analytics --typescript --tailwind --eslint --app --src-di
```

1.2 Dependency Installation

The project requires several key packages, each serving a specific purpose:

Package	Purpose
`prisma` + `@prisma/client`	Database ORM — schema definition, migrations, type-safe queries
`@auth/prisma-adapter`	Connects Auth.js to Prisma for user/account/session storage
`next-auth@beta`	Auth.js v5 — authentication framework for Next.js
`stripe`	Stripe Node.js SDK for server-side payment operations
`@anthropic-ai/sdk`	Official Claude API SDK for AI analysis features
`recharts`	React charting library for data visualizations
`posthog-js`	PostHog client for product analytics and session replay
`papaparse`	CSV parsing — handles large files with streaming
`xlsx`	Excel file parsing (.xlsx, .xls formats)
`react-dropzone`	Drag-and-drop file upload component
`bcryptjs`	Password hashing for credential-based auth
`zod`	Schema validation for API inputs and form data

1.3 shadcn/ui Setup

shadcn/ui is **not a package dependency** — it's a CLI that copies component source code into your project. This gives full control over styling and behavior. Components to install:

`button`, `card`, `input`, `label`, `dialog`, `table`, `dropdown-menu`, `avatar`, `badge`, `tabs`, `toast`, `separator`, `sheet`, `select`, `textarea`, `skeleton`, `progress`

1.4 Prisma Schema

The Prisma schema defines the database structure. After writing it, `npx prisma generate` creates TypeScript types, and `npx prisma db push` syncs the schema to the database.

> **Prisma Workflow**
>
> `schema.prisma` → `prisma generate` (types) → `prisma db push` (sync DB) → `prisma studio` (visual browser)

1.5 Prisma Client Singleton

In development, Next.js hot-reloads create new Prisma instances on each reload, exhausting database connections. The singleton pattern stores the client on `globalThis` to reuse across reloads:

```
// src/lib/prisma.ts
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient }
export const prisma = globalForPrisma.prisma || new PrismaClient()
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

1.6 Middleware

Next.js middleware runs **before every request** at the edge. It handles:

- **Auth protection**: Redirects unauthenticated users from `/dashboard/*`, `/analyze/*`, etc. to `/login`
- **Admin guard**: Checks JWT role claim for `/admin/*` routes
- **Public routes**: Allows unrestricted access to `/`, `/pricing`, `/login`, `/signup`, `/api/stripe/webhook`

1.7 Environment Variables

The `.env.example` file documents all required environment variables:

```
# Database
DATABASE_URL="postgresql://..."

# Auth
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="generate-with-openssl-rand-base64-32"
GOOGLE_CLIENT_ID=""
```

```
GOOGLE_CLIENT_SECRET=""

# Stripe
STRIPE_SECRET_KEY=""
STRIPE_WEBHOOK_SECRET=""
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=""
STRIPE_PRO_PRICE_ID=""
STRIPE_ENTERPRISE_PRICE_ID=""

# AI
ANTHROPIC_API_KEY=""

# Analytics
NEXT_PUBLIC_GTM_ID=""
NEXT_PUBLIC_POSTHOG_KEY=""
NEXT_PUBLIC_POSTHOG_HOST="https://us.i.posthog.com"

# Storage
BLOB_READ_WRITE_TOKEN=""
```

Phase 2: Authentication System

~8 files · Auth.js v5, login, signup, password reset, user menu

2.1 Auth.js v5 Configuration

Auth.js v5 (the successor to NextAuth.js) is configured in `src/lib/auth.ts`. It exports the `auth` function (for server-side session checks), `handlers` (for API routes), and `signIn` / `signOut` functions.

Authentication Providers

Provider	Use Case	How It Works
Google OAuth	One-click social login	User clicks "Continue with Google" → OAuth flow → Account record created → JWT issued
Credentials	Email + password login	User submits form → bcrypt compare against stored hash → JWT issued on match
Email (Magic Link)	Passwordless login	User enters email → verification link sent → click link → auto-login with JWT

JWT vs Database Sessions

We use **JWT strategy** instead of database sessions because: (1) Serverless functions are stateless — JWT carries all session data in the cookie, (2) No database round-trip on every request, (3) Edge middleware can read JWTs without a database connection. Trade-off: JWTs can't be individually revoked (only by changing the secret, which revokes all).

2.2 Auth Callbacks

Auth.js callbacks customize the JWT and session data flow:

- **jwt callback**: Adds `user.id` and `user.role` to the JWT token on sign-in
- **session callback**: Exposes `token.sub` (user ID) and `token.role` on the client-side session object

This means any component calling `useSession()` or `auth()` gets the user's role, enabling admin checks without a database query.

2.3 Login Page

Features: Google OAuth button, email/password form with Zod validation, "Forgot password?" link, "Don't have an account?" link to signup. Uses shadcn/ui Card component for the form container.

2.4 Signup Page

Features: Name, email, password fields with real-time Zod validation (minimum 8 chars, email format). On submit, password is bcrypt-hashed and stored. User automatically gets FREE subscription tier.

2.5 Forgot Password Page

Sends a magic link to the user's email for password reset. In production, integrates with Resend or SendGrid for email delivery.

2.6 UserMenu Component

Dropdown menu in the app topbar showing: user avatar/initials, name, email, links to Settings and Billing, and Sign Out action. Uses shadcn/ui DropdownMenu.

Phase 3: Marketing Site

~10 files · Landing page, pricing, responsive layout, dark mode

3.1 Landing Page Structure

The landing page is the primary conversion funnel. It follows a proven SaaS layout pattern:

1. **Hero Section** — Bold headline, supporting subtext, primary CTA button, optional product screenshot/mockup
2. **Social Proof Bar** — "Trusted by 1000+ analysts" or logo strip of companies
3. **Features Grid** — 6 feature cards with icons, titles, and descriptions
4. **How It Works** — 3-step visual flow: Upload → Analyze → Report
5. **Testimonials** — 3 customer quotes with names and roles
6. **CTA Section** — Final call-to-action before footer

3.2 Feature Cards

Feature	Description
AI-Powered Analysis	Ask questions about your data in plain English. Get insights, patterns, and anomalies instantly.
Smart Visualizations	AI recommends the best chart types. Interactive graphs that tell your data's story.
Multi-Format Upload	Drop CSV, Excel, or JSON files. Auto-detection of schemas, types, and structure.
Natural Language Queries	No SQL required. Ask "What's the trend in Q4 sales?" and get an answer.
Report Generation	Compile analyses into shareable reports. Export as PDF or share via link.
Enterprise Security	SOC2 ready. Data encryption at rest and in transit. RBAC and audit logging.

3.3 Pricing Page

Three-tier pricing with feature comparison. Each tier has a Stripe checkout button that creates a checkout session and redirects the user.

3.4 Marketing Layout

Shared across all marketing pages:

- **Navbar**: Logo, nav links (Features, Pricing), Login button, "Get Started" CTA, mobile hamburger menu
- **Footer**: Logo, product links, legal links (Privacy, Terms), social links, copyright
- **Theme Toggle**: Light/dark mode switch using next-themes

> **SEO Strategy**
>
> Marketing pages use Next.js `generateMetadata()` for per-page SEO. Each page gets: unique title, description, Open Graph image, Twitter card, canonical URL. The landing page targets keywords like "AI data analytics", "analyze CSV with AI", "natural language data analysis". Pricing page targets "data analytics pricing".

Phase 4: Application Dashboard

~12 files · Sidebar layout, dashboard, datasets, AI workspace, reports, settings, billing

4.1 App Layout (Sidebar + Topbar)

The app shell provides consistent navigation for all protected pages:

- **Sidebar** (left, 256px): Logo, nav links (Dashboard, Datasets, Analyze, Reports), bottom section (Settings, Billing, Sign Out). Collapsible on mobile using shadcn Sheet.
- **Topbar** (top): Page title, search input, notification bell, UserMenu dropdown
- **Main Content**: Right of sidebar, below topbar. Scrollable container with max-width constraint.

4.2 Dashboard Page

The first thing users see after login. Provides an at-a-glance overview:

Stats Cards

4 metric cards: Total Datasets, AI Queries Used, Reports Generated, Storage Used. Each shows current value and usage bar (vs. plan limit).

Recent Activity

Chronological list of recent actions: uploads, queries, report generations. Shows action type icon, description, and timestamp.

Quick Actions

Large action cards: "Upload Dataset", "Start Analysis", "View Reports". Reduce clicks to reach key workflows.

Usage Chart

Recharts area chart showing AI query usage over the current billing period. Helps users understand their consumption pattern.

4.3 Datasets Page

Central hub for data management. Two main areas:

- **Upload Section**: react-dropzone powered drag-and-drop area. Accepts CSV, XLSX, JSON. Shows upload progress, validates file size against plan limits, parses file on upload to detect schema.
- **Dataset Table**: Lists all user datasets with columns: Name, Format, Rows, Size, Created Date, Actions (Analyze, Preview, Delete). Sortable and paginated.

Clicking a dataset opens a **Preview Modal** showing the first 100 rows in a data table with column type badges.

4.4 Analyze Page (AI Workspace)

The core product experience — a chat-style interface for data analysis:

1. **Dataset Selector**: Dropdown to choose which dataset to analyze
2. **Chat Interface**: User types natural language queries (e.g., "What are the top 5 products by revenue?")
3. **AI Response Area**: Claude's response renders with:
   - Text insights (markdown formatted)
   - Auto-generated charts (Recharts, configured by AI)
   - Data tables (for tabular results)
   - "Save to Report" button on each response
4. **Conversation History**: Full chat history preserved during session, enabling follow-up questions

> **How AI Analysis Works**
>
> User query + dataset (first 500 rows + full schema) are sent to Claude API. The prompt instructs Claude to return structured JSON with: `insights[]` (text findings), `charts[]` (Recharts-compatible config objects), and `summary` (one-paragraph overview). The frontend parses this JSON and renders each section with appropriate components.

4.5 Reports Page

Gallery view of all generated reports. Each report card shows: title, source dataset, creation date, number of charts/insights. Click to view full report. Export as PDF button. Delete action.

4.6 Settings Page

User profile management: edit name, change password, notification preferences, theme selection, API key display (for future API access). Uses form with Zod validation and optimistic UI updates.

4.7 Billing Page

Subscription management powered by Stripe:

- Current plan display with feature limits
- Usage meters (queries, datasets, storage)

- "Manage Subscription" button → Stripe Customer Portal (upgrade, downgrade, cancel, update payment)
- Invoice history table with download links

Phase 5: API Routes

~10 files · Stripe, AI, datasets, admin endpoints

5.1 API Route Architecture

All API routes are Next.js Route Handlers (serverless functions). They follow a consistent pattern:

```
export async function POST(request: Request) {
  // 1. Authenticate — get session from Auth.js
  const session = await auth()
  if (!session) return Response.json({ error: 'Unauthorized' }, { status: 401 })

  // 2. Validate — parse request body with Zod
  const body = schema.parse(await request.json())

  // 3. Authorize — check subscription limits
  // 4. Execute — perform business logic
  // 5. Audit — log the action
  // 6. Return — JSON response
}
```

5.2 Stripe Routes

POST /api/stripe/checkout

Creates a Stripe Checkout Session for plan upgrades. Receives `priceId` in the body. Creates a Stripe customer if one doesn't exist, then generates a checkout URL. Returns the URL for client-side redirect.

POST /api/stripe/portal

Creates a Stripe Customer Portal session for subscription management. The portal is hosted by Stripe and handles: plan changes, payment method updates, invoice viewing, and cancellation. Returns the portal URL.

POST /api/stripe/webhook

Receives Stripe webhook events. **Critical: This route must NOT use auth middleware** — it's called by Stripe's servers, not by authenticated users. It verifies the webhook signature using the webhook secret.

Event	Action
`checkout.session.completed`	Create/update Subscription record, set plan to PRO or ENTERPRISE
`customer.subscription.updated`	Sync plan and status changes (upgrades, downgrades)
`customer.subscription.deleted`	Revert user to FREE plan
`invoice.payment_succeeded`	Reset queriesUsed counter for new billing period
`invoice.payment_failed`	Set subscription status to "past_due", send notification

Webhook Idempotency

Stripe may send the same webhook event multiple times. The webhook handler should be **idempotent** — processing the same event twice should not create duplicate records or charge users twice. We achieve this by checking if the subscription already matches the event data before updating.

5.3 AI Routes

POST /api/ai/analyze

Accepts `datasetId` and `prompt`. Fetches the dataset from storage, sends to Claude with a structured analysis prompt, returns insights + chart configs. Checks subscription query limits before processing. Increments `queriesUsed` on success.

POST /api/ai/query

Natural language query against a dataset. Claude converts the question into data filtering/aggregation operations, executes them against the dataset, and returns formatted results. Supports follow-up queries with conversation context.

5.4 Dataset Routes

GET /api/datasets

Lists all datasets for the authenticated user. Returns id, name, format, rowCount, columns, createdAt.

POST /api/datasets/upload

Handles multipart file upload. Process: receive file → validate format and size → upload to Vercel Blob → parse file to detect schema → create Dataset record with metadata → return dataset info.

DELETE /api/datasets/[id]

Deletes dataset record and associated file from Vercel Blob. Also cascades to delete related reports.

5.5 Admin Routes

Protected by admin role check. `GET /api/admin/users` returns paginated user list with subscription info. `GET /api/admin/stats` returns platform metrics: total users, MRR, active subscriptions by plan, API usage totals.

Phase 6: AI Engine

~3 files · Claude API wrapper, file parser, usage tracking

6.1 Claude API Service (src/lib/ai.ts)

Central AI service that wraps all Claude API interactions. Four main functions:

analyzeDataset(data, prompt)

Sends the dataset schema + sample rows + user prompt to Claude. The system prompt instructs Claude to return structured JSON:

```
{
  "insights": [
    { "title": "Revenue Trend", "description": "Revenue increased 23% QoQ...", "importa
  ],
  "charts": [
    {
      "type": "bar",
      "title": "Revenue by Quarter",
      "data": [{ "quarter": "Q1", "revenue": 45000 }, ...],
      "xKey": "quarter",
      "yKey": "revenue"
    }
  ],
  "summary": "The dataset shows strong growth with seasonal patterns..."
}
```

This structure maps directly to Recharts components on the frontend.

generateChartConfig(data, prompt)

AI recommends the optimal visualization for given data. Returns chart type (bar, line, pie, scatter, area), axis mappings, color scheme, and aggregation method.

queryData(data, query)

Converts natural language to data operations. Claude analyzes the schema, determines the right filter/group/aggregate operations, and returns results. Example: "Show me average salary by department" → groups by department column, averages salary column.

generateReport(data, analyses)

Takes multiple previous analysis results and compiles them into a cohesive report structure with executive summary, key findings, detailed charts, and recommendations.

> **Prompt Engineering Strategy**
>
> All prompts use Claude's system prompt for role definition ("You are a data analyst..."), include the full dataset schema in the user message, and use structured output formatting instructions. The system prompt specifies the exact JSON structure expected, and Claude's instruction-following capability ensures consistent, parseable responses.

6.2 File Parser (src/lib/upload.ts)

Handles parsing of uploaded files into a standardized internal format:

Format	Parser	Features
CSV	papaparse	Auto-detects delimiter, handles quoted fields, streaming for large files, header row detection
Excel (.xlsx)	xlsx (SheetJS)	Multi-sheet support (uses first sheet), date format detection, formula evaluation
JSON	Native JSON.parse	Supports array of objects and nested structures, auto-flattens one level deep

After parsing, the system performs **schema detection**:

- Column name extraction from headers
- Type inference: string, number, date, boolean (samples first 100 rows)
- Null/missing value counting per column
- Preview generation (first 100 rows for quick display)

6.3 Usage Tracking & Rate Limiting

Each subscription tier has defined limits. Before any AI operation:

1. Fetch user's Subscription record
2. Check `queriesUsed < queriesLimit`
3. If limit reached, return 429 with upgrade prompt
4. On success, increment `queriesUsed` atomically
5. Log to AuditLog for tracking

Phase 7: Analytics & Marketing Pixels

~6 files · GTM, PostHog, unified events, conversion tracking

7.1 Analytics Architecture

The analytics system uses a dual-track approach: GTM for marketing attribution and PostHog for product analytics.

```
User Action
    |
    v
track('event', data)
    |
    +---> GTM dataLayer.push()       +---> posthog.capture()
    |         |                       |
    |    GTM Console routes to:       PostHog Dashboard:
    |    ├── GA4 (all events)         ├── Funnels
    |    ├── Meta Pixel (conversions) ├── Session replay
    |    ├── LinkedIn (B2B events)    ├── Feature flags
    |    └── Twitter/X (conversions)  └── A/B testing
    |
    v
Single track() call powers everything
```

7.2 GTM Integration

Google Tag Manager acts as a **container** for all marketing pixels. Two script tags are added to the root layout:

- `<script>` in `<head>` : GTM JavaScript snippet
- `<noscript>` in `<body>` : Fallback iframe for users with JS disabled

All ad platform pixels (GA4, Meta, LinkedIn, Twitter/X) are configured **inside the GTM web console**, not in application code. This means adding or removing a pixel requires zero code changes and zero deployments.

7.3 PostHog Integration

PostHog client initializes in a React context provider wrapping the entire app. It provides:

- **Auto-capture**: Automatically tracks clicks, page views, and form submissions
- **Session replay**: Records user sessions for UX debugging (configurable sampling rate)
- **Feature flags**: Server-side and client-side feature flag evaluation

- **A/B testing**: Built on feature flags, integrates with analytics for experiment analysis
- **Custom events**: Via `posthog.capture()` for product-specific tracking

7.4 Unified Event System

A single `track()` function dispatches events to both GTM and PostHog simultaneously:

```
// src/lib/analytics/events.ts
export function track(event: string, properties?: Record<string, any>) {
  // GTM
  window.dataLayer?.push({ event, ...properties })
  // PostHog
  posthog?.capture(event, properties)
}
```

Key Events Tracked

Event	When	Properties
`signup_completed`	User creates account	method (google/email)
`dataset_uploaded`	File upload succeeds	format, rowCount, fileSize
`ai_query_sent`	User sends analysis query	datasetId, promptLength
`report_generated`	Report saved	chartCount, insightCount
`checkout_started`	User clicks upgrade	plan, price
`plan_upgraded`	Stripe confirms payment	fromPlan, toPlan, value
`page_view`	Route change	path, referrer

7.5 GTM Container Setup Guide

After deploying the application, configure these tags in the GTM web console:

1. **GA4 Configuration Tag**: Trigger on All Pages, measurement ID from GA4 property
2. **GA4 Event Tag**: Trigger on custom events, forward event name and properties
3. **Meta Pixel**: Base code tag (All Pages) + conversion event tags (signup, purchase)
4. **LinkedIn Insight Tag**: Partner ID tag (All Pages) + conversion tags
5. **Twitter/X Universal Website Tag**: Pixel ID (All Pages) + conversion events

Phase 8: Admin Panel

~4 files · Admin dashboard, user management, access control

8.1 Admin Access Control

Admin routes are protected at two levels:

1. **Middleware level**: The Next.js middleware checks the JWT `role` claim. Requests to `/admin/*` from non-admin users receive a 403 redirect.
2. **API level**: Admin API routes (`/api/admin/*`) independently verify the session role, providing defense in depth.

8.2 Admin Dashboard

Displays platform-wide metrics for business operations:

- **Key Metrics**: Total users, MRR (Monthly Recurring Revenue), active subscriptions count, total AI queries processed
- **Subscription Distribution**: Pie chart showing Free vs Pro vs Enterprise users
- **Revenue Chart**: Line chart showing MRR growth over time
- **Recent Signups**: Table of newest users with signup date and method
- **API Usage**: Bar chart of daily AI query volume

8.3 User Management

Comprehensive user administration interface:

- **User Table**: Searchable, paginated list with columns: Name, Email, Role, Plan, Status, Created, Actions
- **Search**: Filter by name, email, plan, or role
- **Actions**: View user details, change role (USER/ADMIN), disable account, view usage history
- **User Detail Panel**: Slide-over panel showing: subscription details, dataset list, query history, audit log entries

Phase 9: Polish & Deployment

~5 files · Loading states, error handling, SEO, deployment

9.1 Loading States

Every page and data-fetching component gets appropriate loading UX:

- **Page-level**: Next.js `loading.tsx` files show skeleton screens during route transitions
- **Component-level**: shadcn Skeleton components for cards, tables, and charts during data fetch
- **Action-level**: Button loading spinners during form submissions and API calls

9.2 Error Handling

- **Error Boundaries**: Next.js `error.tsx` files catch rendering errors per route segment
- **API Error Responses**: Consistent JSON error format with status codes and user-friendly messages
- **Toast Notifications**: shadcn Toast for success/error feedback on user actions
- **Not Found**: Custom `not-found.tsx` page with helpful navigation

9.3 SEO Optimization

Marketing pages are optimized for search engines:

- **Metadata**: Per-page title, description via Next.js `generateMetadata()`
- **Open Graph**: OG image, title, description for social sharing (Facebook, LinkedIn)
- **Twitter Cards**: Large image summary cards for Twitter/X sharing
- **Structured Data**: JSON-LD for SoftwareApplication schema (helps Google understand the product)
- **Sitemap**: Auto-generated via Next.js `sitemap.ts`
- **Robots.txt**: Allow marketing pages, disallow app/admin routes

9.4 Deployment to Vercel

1. Connect GitHub repository to Vercel
2. Configure environment variables in Vercel dashboard
3. Set up Neon database and run migrations
4. Configure Stripe webhook endpoint to point to production URL
5. Set up GTM container with production pixel IDs

6. Deploy via `git push` — Vercel auto-builds and deploys

14. Pricing & Business Model

Feature	Free	Pro ($29/mo)	Enterprise ($99/mo)
Datasets	3	50	Unlimited
AI Queries / Month	10	500	Unlimited
Max File Size	5 MB	50 MB	500 MB
Reports	5	Unlimited	Unlimited
Export Formats	CSV	CSV, PDF	CSV, PDF, API
Support	Community	Email	Priority
API Access	No	No	Yes
Team Members	1	1	5
Custom Branding	No	No	Yes

Revenue Model

The Free tier drives user acquisition (try before you buy). Pro ($29/mo) is the primary revenue driver, targeting individual analysts. Enterprise ($99/mo) targets teams with higher limits and API access. At 100 paying users (70 Pro + 30 Enterprise), MRR = $5,000. At 1,000 paying users, MRR = $40,000+.

15. Data Flow Architecture

File Upload Flow

```
User drops file
    |
    v
Frontend: validate format + size against plan limits
    |
    v
POST /api/datasets/upload (multipart FormData)
    |
    v
```

```
Server: parse file (papaparse/xlsx/JSON.parse)
    |
    v
Detect schema: columns, types, row count
    |
    v
Upload raw file to Vercel Blob --> fileUrl
    |
    v
Create Dataset record in Prisma (metadata + fileUrl)
    |
    v
Return dataset info to frontend
    |
    v
Frontend: show in dataset table + success toast
```

AI Analysis Flow

```
User types question in chat
    |
    v
POST /api/ai/analyze { datasetId, prompt }
    |
    v
Server: auth check --> subscription limit check
    |
    v
Fetch dataset from Vercel Blob
    |
    v
Parse dataset, prepare context (schema + sample rows)
    |
    v
Send to Claude API with structured output prompt
    |
    v
Claude returns JSON: { insights[], charts[], summary }
    |
    v
Increment queriesUsed, log to AuditLog
    |
    v
Return structured response to frontend
    |
    v
Frontend: render insights as text, charts via Recharts,
          summary as markdown, "Save to Report" button
```

Billing Flow

```
User clicks "Upgrade to Pro"
    |
    v
POST /api/stripe/checkout { priceId }
    |
    v
Create Stripe Checkout Session (or Customer if needed)
    |
    v
Redirect user to Stripe-hosted checkout page
    |
    v
User completes payment on Stripe
    |
    v
Stripe sends webhook: checkout.session.completed
    |
    v
POST /api/stripe/webhook (signature verified)
    |
    v
Update Subscription: plan=PRO, limits=500 queries
    |
    v
User returns to /billing, sees updated plan
```

16. Security Considerations

Threat	Mitigation
Unauthorized access	Edge middleware auth check on all protected routes; JWT verification on API routes
Privilege escalation	Role check at middleware + API level (defense in depth); role stored in JWT, verified server-side
Data leakage	All dataset queries filter by userId — users can only access their own data
File upload attacks	Allowlist of file extensions (csv, xlsx, json); file size limits per plan; content-type validation
Stripe webhook forgery	Verify webhook signature using STRIPE_WEBHOOK_SECRET before processing
AI prompt injection	Dataset content is treated as data (user message), not instructions (system prompt); output is JSON-parsed, not executed
XSS	React's default escaping; AI output rendered as markdown (sanitized), not raw HTML
CSRF	Auth.js built-in CSRF token validation on all auth actions
SQL injection	Prisma ORM uses parameterized queries — no raw SQL
Brute force	Rate limiting on auth endpoints; account lockout after repeated failures
Data at rest	Neon encrypts data at rest (AES-256); passwords bcrypt-hashed with salt
Data in transit	HTTPS enforced by Vercel; all API calls use TLS

17. Verification & Testing Checklist

Functional Tests

#	Test	Expected Result
1	Visit / (landing page)	Hero, features, testimonials render; CTA buttons link to /signup

#	Test	Expected Result
2	Visit /pricing	3 pricing tiers display; checkout buttons work for logged-in users
3	Sign up with email	Account created, redirect to /dashboard, FREE subscription auto-created
4	Sign in with Google	OAuth flow completes, Account linked, redirect to /dashboard
5	Visit /dashboard without auth	Redirect to /login
6	Upload CSV file	File parsed, schema detected, Dataset record created, appears in table
7	Upload file exceeding size limit	Error message with plan upgrade suggestion
8	Send AI analysis query	Claude response with insights + charts renders correctly
9	Exceed query limit (Free tier)	429 response with "Upgrade to Pro" prompt
10	Save analysis to report	Report created, appears in /reports gallery
11	Click "Upgrade to Pro"	Stripe checkout page opens, test card works, webhook processes
12	Manage subscription	Stripe portal opens, can change plan or cancel
13	Admin user visits /admin	Admin dashboard with platform metrics
14	Non-admin visits /admin	Redirect to /dashboard (access denied)
15	Toggle dark mode	All pages switch themes correctly
16	Mobile viewport	Responsive layout, hamburger menu, touch-friendly

Analytics Verification

1. Open GTM Debug/Preview mode — verify dataLayer pushes on page views and custom events
2. Check PostHog dashboard — verify events appear with correct properties
3. Open browser DevTools Network tab — verify GA4, Meta Pixel, LinkedIn, Twitter/X requests fire
4. Test conversion events: signup → verify purchase event in all platforms

End of Plan Document

AI Data Analytics SaaS — Complete Build Plan

Generated February 18, 2026